

Python Coding for AI

Ch1: Python의 직관적 이해

Ch2: Python OOP

Ch3: What is Numpy & Why Numpy?

Ch4: What is Pandas & Why Pandas?

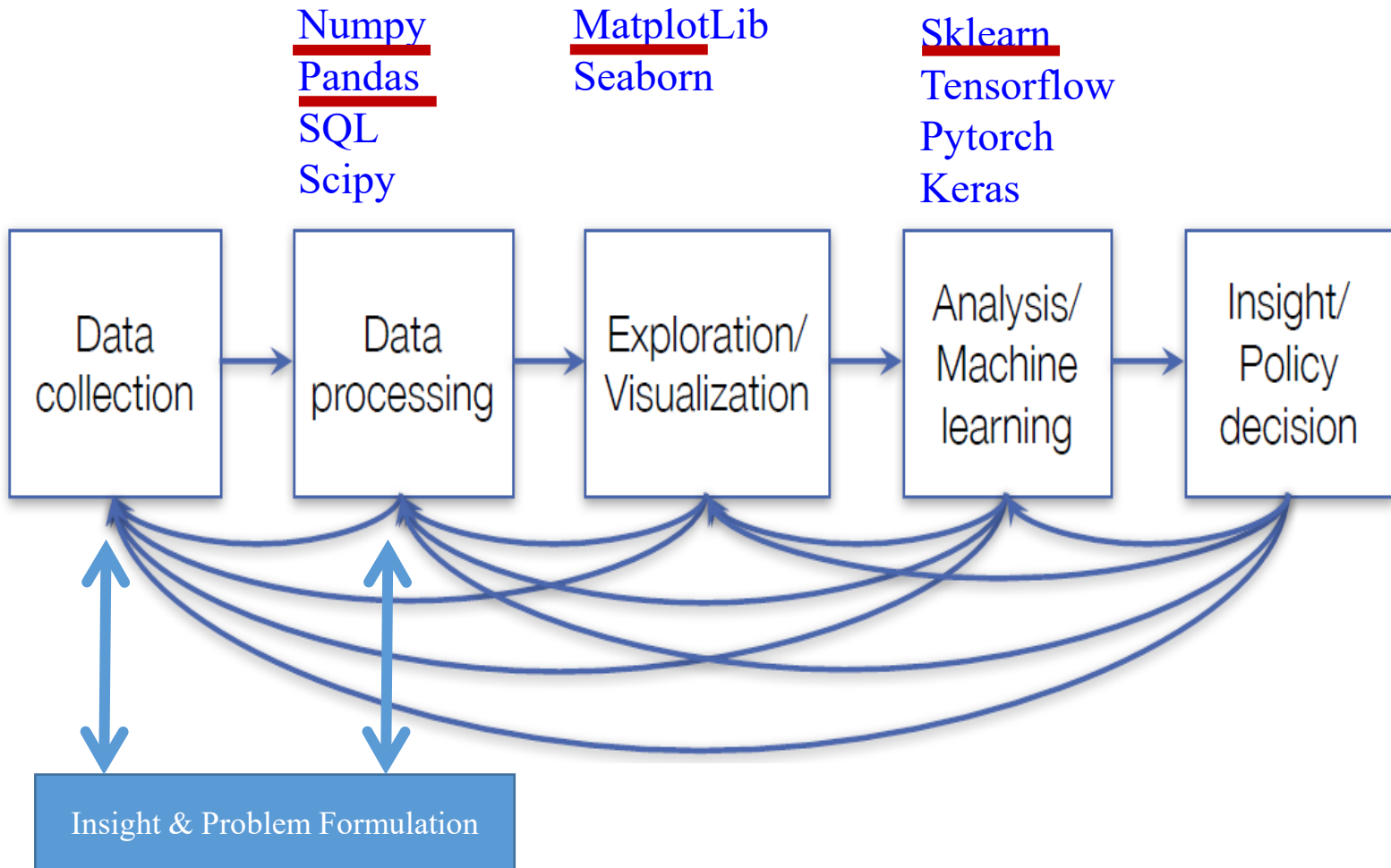
Ch5: What is Matplotlib & Why Matplotlib?

Ch6: What is Sklearn & Why Sklearn?

Ch7: What is Data Preprocessing & Why Data Preprocessing?

서울대 컴퓨터공학부
김형주 명예교수

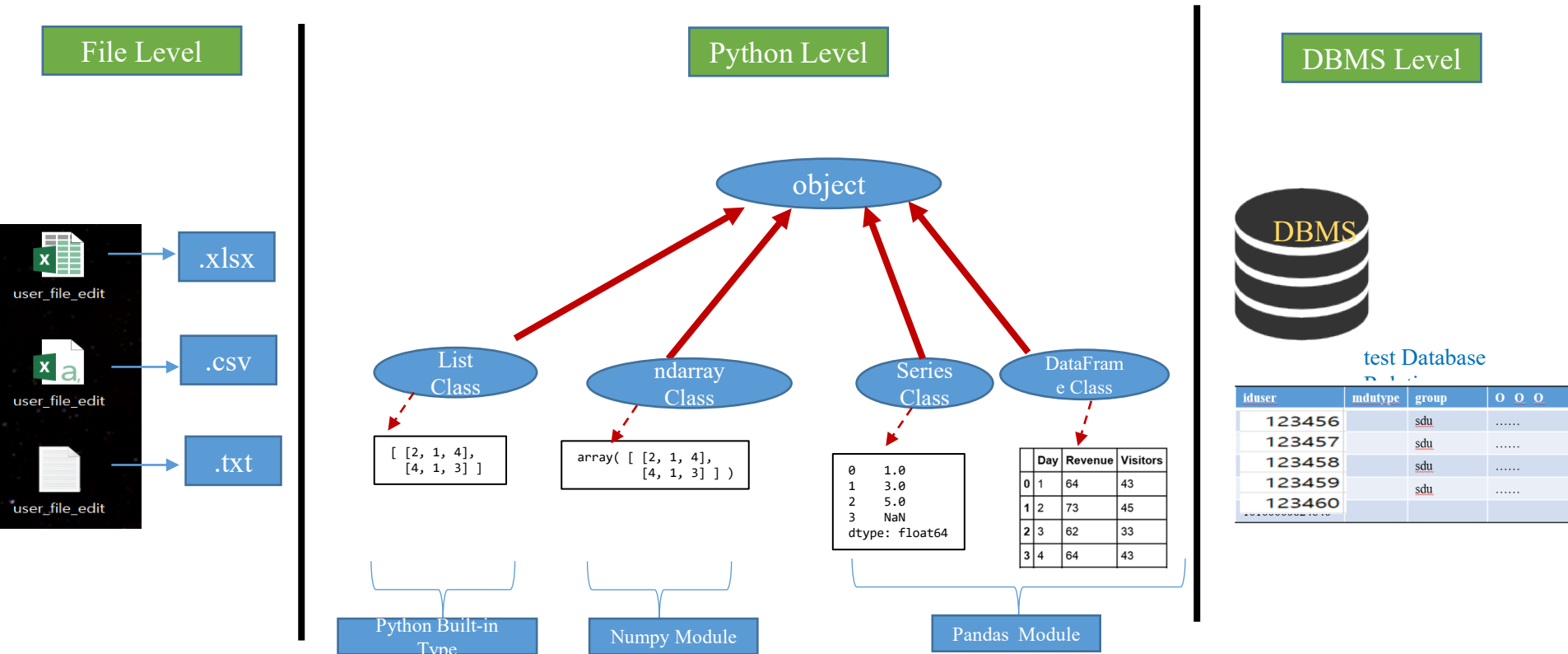
Major Steps of Data Analytics



Data Preprocessing with Python

- Introduction
- Data Cleaning
 - Solutions for Missing Values / Data Inconsistency / Outliers
- Table Integration
- Data Transformation
 - Data Scaling
 - Vector Normalization
- Data Reduction & Resampling
 - Row & Column Reduction
 - Imbalanced Data
- Data Discretization
 - Encoding

Various Kinds of Data: Therefore, Data Preprocessing Anywhere!



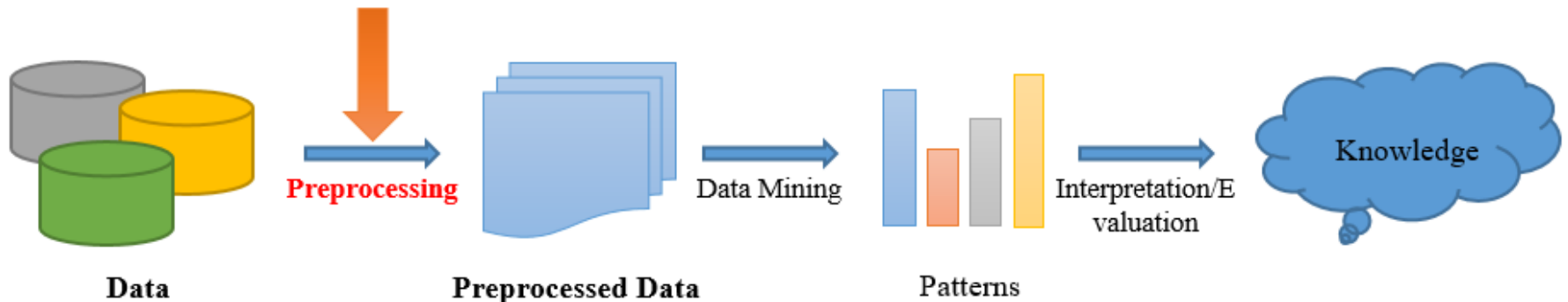
Data Preprocessing (데이터 전처리) [1/2]

- 정리안된 Raw Data를 분석 가능한 상태로 만드는 반복적이고 지루한 작업
- 전체 데이터분석작업의 60%~ 80% 시간이 데이터 전처리에 투입
- 데이터 분석 결과 및 모델 성능에 직접적 영향

Data scientist 들은 시간을 어디에 사용하나?

- 데이터 수집: 19%
- 데이터 클리닝: 60%
- 학습데이터 확보: 3%
- 패턴확보 작업: 9%
- 알고리즘 수정: 4%
- 기타: 5%

Data Preprocessing



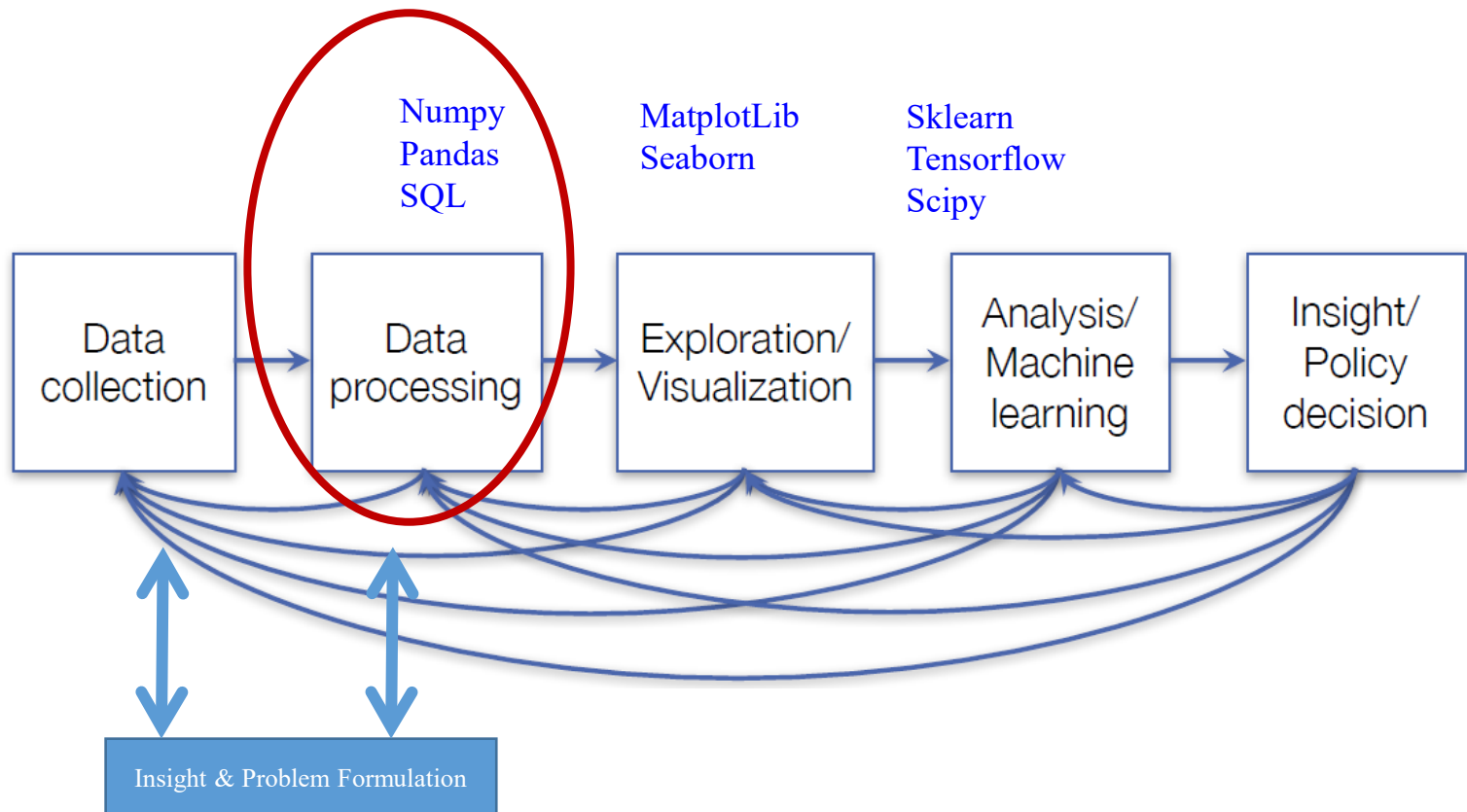
Data Preprocessing (데이터 전처리) [2/2]

• 데이터 전처리 6요소

Data Loading (데이터 불러오기)	- Move data from source to work space - Example: load text file in python
Data Cleaning (데이터 클리닝)	- Fill in missing values (결측치 대체) - Smooth noisy data - Identify or remove outliers (이상치 확인과 제거)
Data Integration (데이터 통합)	- Integration of multiple databases, data cubes or files (다양한 로그파일/데이터베이스 통합/ 일관성 있는 데이터 형태로 변환)
Data Transformation (데이터 변환)	- Normalization / Aggregation / Summarization (정규화) / (집합화) / (요약)
Data Reduction (데이터 축소)	- Obtains reduced representation in volume but produces the same or similar analytical results - (컴퓨팅 시간 등 고려 위해 데이터 축소)
Data Discretization (데이터 이산화)	- Part of data reduction but with particular importance, especially for numerical data - Convert categorical values to numerical values (속성 값을 수치 값으로 변환)

* S. Zhang, C. Zhang, and Q. Yang, "Data preparation for data mining," Applied Artificial Intelligence, Vol. 17, No. 5-6, pp. 75-381, 2003.

Major Steps of Data Science



Machine Learning전에 Data Preprocessing이 필요한 상황들! [1/2]

Age	Sex	BMI	ABP
28	남	21.5%	80..120
72	여	19%	70..115
55	남	3%	60..90



Age	Sex	BMI	ABP
0.038	0.051	0.062	0.022
-0.002	-0.045	-0.051	-0.026
0.085	0.051	0.044	-0.006
-0.089	-0.045	-0.012	-0.037

원본 데이터:

	Product	Price	Quantity
0	A	10	100
1	B	20	50
2	C	30	80
3	D		120
4	E	50	

unknown

missing

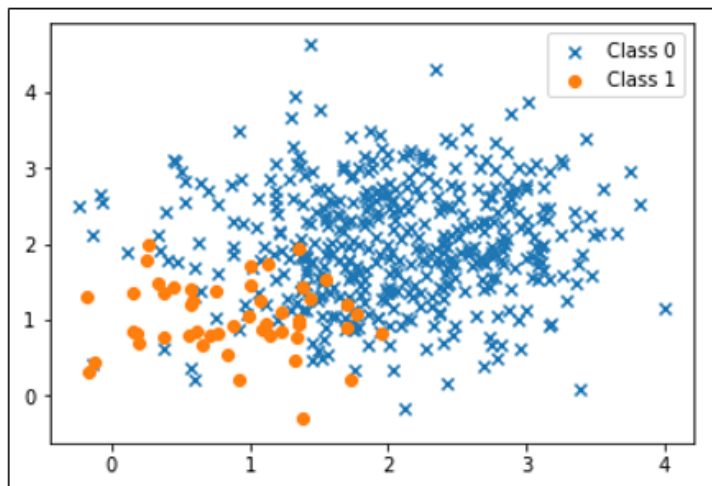


데이터 Cleaning 처리 후:

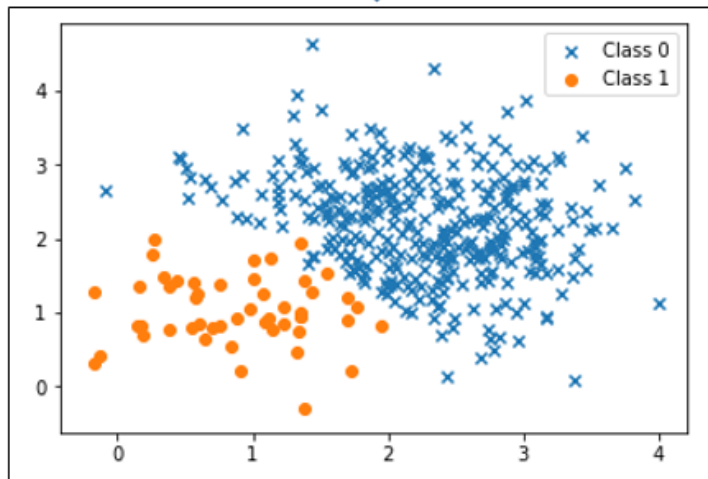
	Product	Price	Quantity
0	A	10.0	100
1	B	20.0	50
2	C	30.0	80
3	D	NaN	120
4	E	50.0	0

Machine Learning전에 Data Preprocessing 필요한 상황들! [2/2]

Imbalanced Data!



Balanced Data!



Season	Season	Season
Spring	1	[1, 0, 0, 0]
Summer	2	[0, 1, 0, 0]
Fall	3	[0, 0, 1, 0]
Winter	4	[0, 0, 0, 1]
Summer	2	[0, 1, 0, 0]

데이터 분석을 위한 다양한 데이터 Format 들

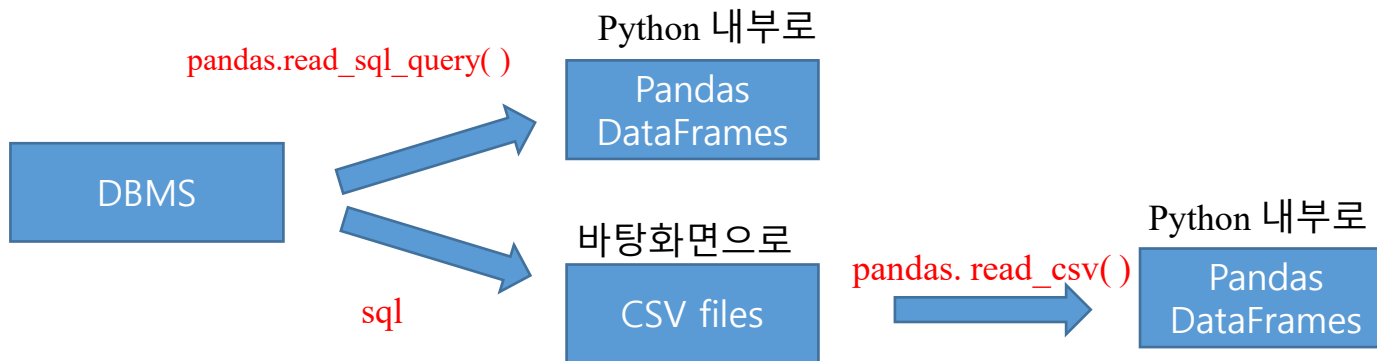
- Numpy ndarray
- Pandas DataFrame
- CSV files
- DBMS Relations
- Web Pages
- Many others

Data Preprocessing은 어떤 format에서도 발생할 수 있다!

그러나 Numpy ndarray와 Pandas DataFrame이 가장 빈번하게 Data Preprocessing의 대상이 된다!

Ubiquitous (어디에나 있는) Pandas Data Frames

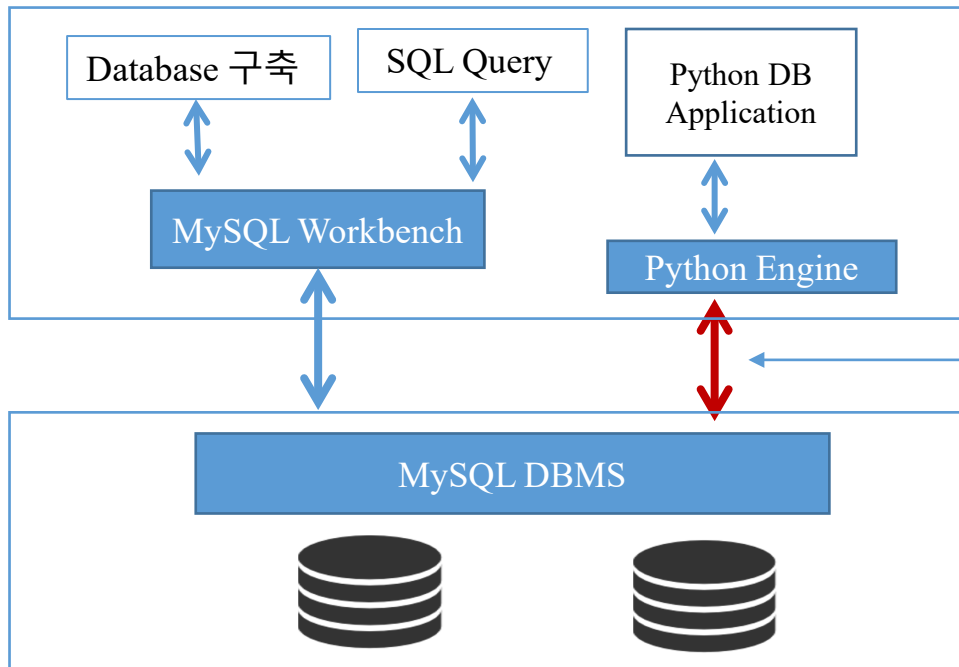
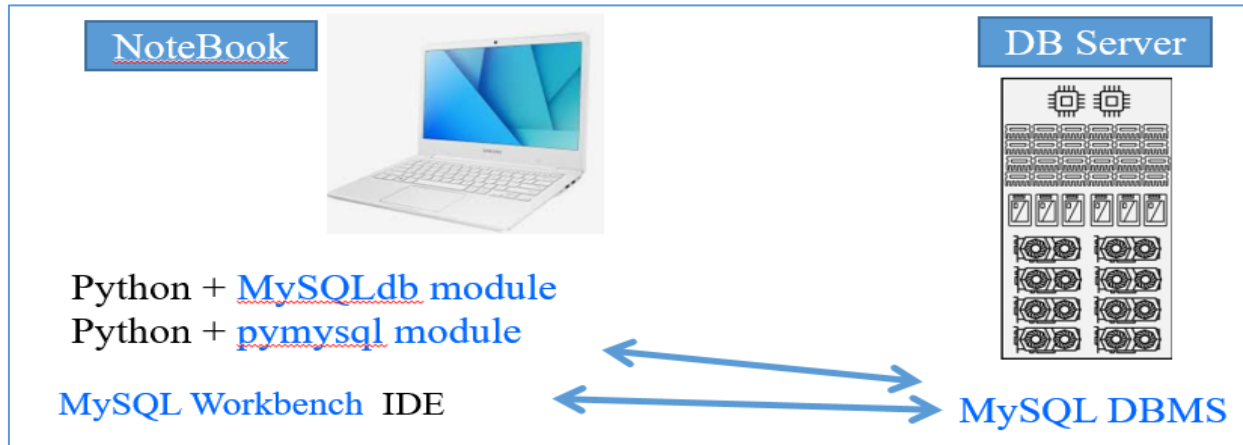
- From Numpy ndarray To Pandas Data Frames
 - 이미 매우 익숙한 내용
- From Database Tables To Pandas Data Frames



- From Web Tables To Pandas Data Frames
 - Text Processing Modules: [BeautifulSoup](#)

MySQL DBMS 와 Python 환경

참고



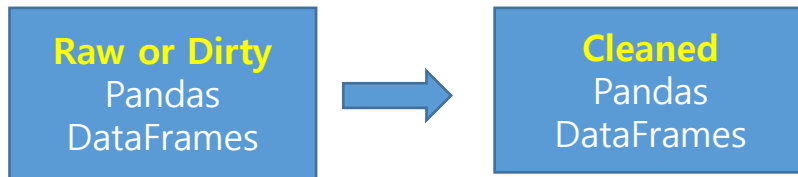
- Python 응용프로그램과 MySQL DBMS가 data를 주고 받으려면 **python-dbms driver (MySQLdb, pymysql)** 를 노트북에 설치해야함
- **python-dbms driver module 사용법을 잘 알아야 함**

Data Preprocessing with Python

- Introduction
- Data Cleaning
 - Solutions for Missing Values / Data Inconsistency / Outliers
- Table Integration
- Data Transformation
 - Data Scaling
 - Vector Normalization
- Data Reduction & Resampling
 - Row & Column Reduction
 - Imbalanced Data
- Data Discretization
 - Encoding

Data Cleaning

- From Raw Data Frames To Cleaned Data Frames
 - Preprocessing with Pandas
 - Preprocessing with SKLearn



- Missing Value 처리
- Data Inconsistency 처리
- Outlier 처리

원본 데이터:

	Product	Price	Quantity
0	A	10	100
1	B	20	50
2	C	30	80
3	D		120
4	E	50	

unknown

missing

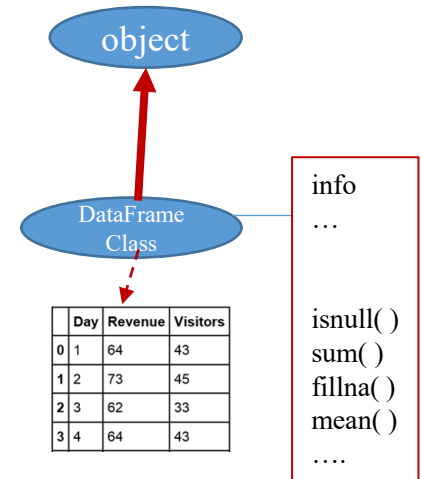
↓

데이터 Cleaning 처리 후:

	Product	Price	Quantity
0	A	10.0	100
1	B	20.0	50
2	C	30.0	80
3	D	NaN	120
4	E	50.0	0

Missing Value Processing in Pandas: fillna()

- DataFrame에서 Missing value 대체 함수
 - DataFrame.fillna(value): NaN 값을 “value”로 대체



'viewCount' column의 missing value들을 'viewCount' column의 평균값으로 대체

```
print(df.viewCount.mean())
```

→ 24.576208358277547

```
df['viewCount'] = df['viewCount'].fillna(df.viewCount.mean())
```

df

iduser	mdutype	group	viewCount
123456	NaN	sdu	12.0
123457	NaN	sdu	23.0
123458	NaN	sdu	4.0
123459	NaN	sdu	6.0
23460	NaN	NaN	NaN

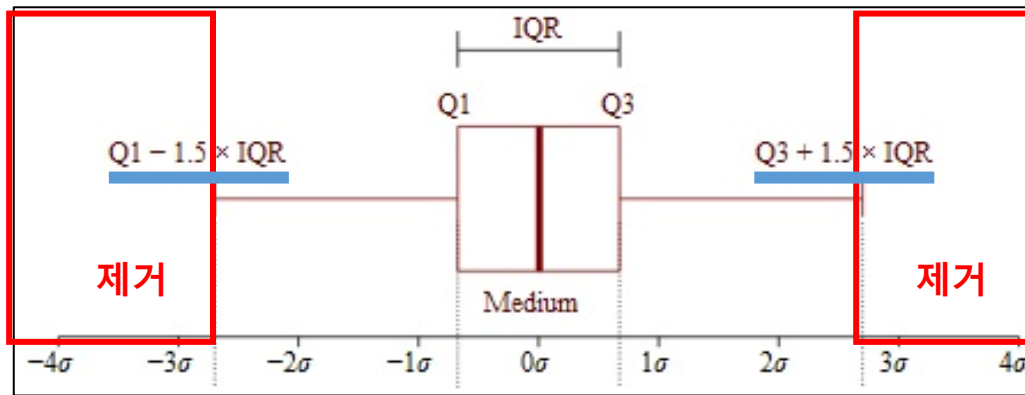


df

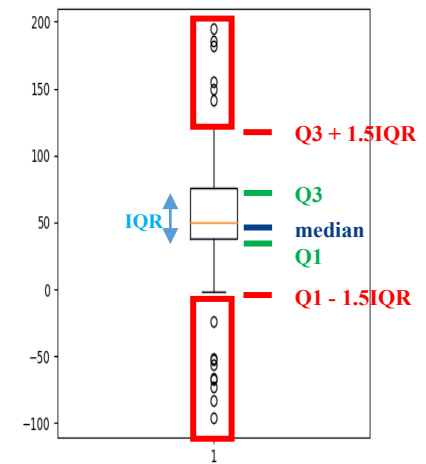
iduser	mdutype	group	viewCount
123456	NaN	sdu	12.0
123457	NaN	sdu	23.0
123458	NaN	sdu	4.0
123459	NaN	sdu	6.0
23460	NaN	NaN	24.576208

Removing Outliers: IQR 방식 [1/3]

- **IQR: InterQuartile Range (사분위 범위) 방식**
 - $1Q = 25\%$ 지역의 값, $3Q = 75\%$ 지역의 값
 - $IQR = 3Q - 1Q$
 - Remove x if $(x < 1Q - 1.5 \times IQR)$ or $(x > 3Q + 1.5 \times IQR)$



plt.boxplot(X)



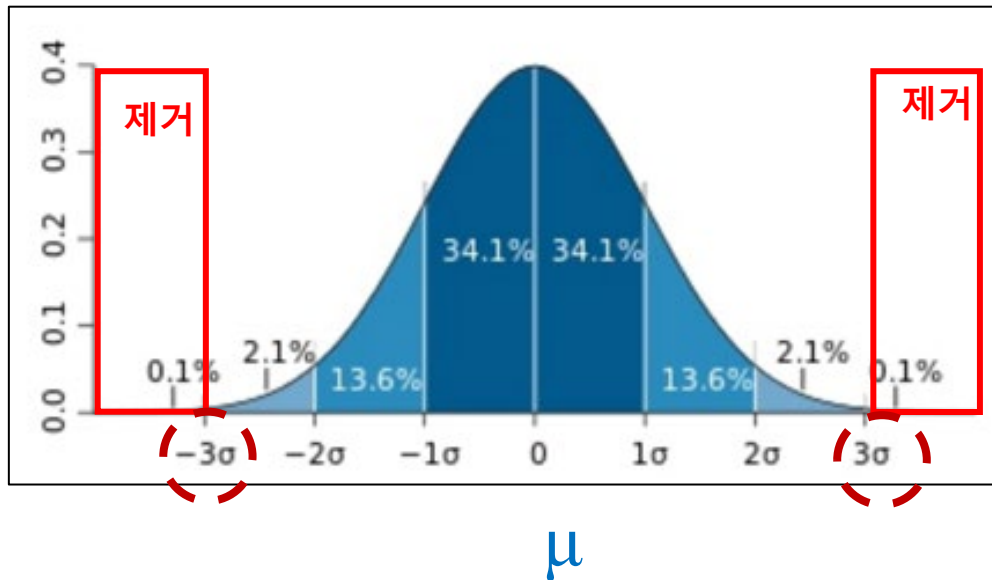
0 quartile = 0 quantile = 0 percentile
1 quartile = 0.25 quantile = 25 percentile
2 quartile = .5 quantile = 50 percentile (median)
3 quartile = .75 quantile = 75 percentile
4 quartile = 1 quantile = 100 percentile

꼭 그렇지는 않지만
Q1는 -3σ 의 근처
Q3는 3σ 의 근처

Removing Outliers: SD (Standard Deviation) 방식 [2/3]

- 표준 편차 방식

- Remove X if $(X > \mu + 3\sigma)$ or $(X < \mu - 3\sigma)$



Removing Outliers: 구현 [3/3]

```
pip install feature-engine
```

```
from feature_engine.outliers import OutlierTrimmer
```

```
# 1. IQR 방식 (IQR * 1.5 기준)
```

```
trimmer_iqr = OutlierTrimmer(capping_method='iqr', tail='both', fold=1.5)
```

```
df_clean = trimmer_iqr.fit_transform(df)
```

```
# 2. SD(표준편차/Gaussian) 방식 (평균 +/- 3*SD 기준) trimmer_sd =  
OutlierTrimmer(capping_method='gaussian', tail='both', fold=3.0)
```

```
df_clean = trimmer_sd.fit_transform(df)
```

Module 이용

```
# 1. Q1, Q3 및 IQR 계산
```

```
Q1 = df['col_name'].quantile(0.25)
```

```
Q3 = df['col_name'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
# 2. 하한선, 상한선 설정
```

```
lower_bound = Q1 - 1.5 * IQR
```

```
upper_bound = Q3 + 1.5 * IQR
```

```
# 3. 조건 만족하는 데이터만 추출 (이상치 제거)
```

```
df_clean = df[(df['col_name'] >= lower_bound) & (df['col_name'] <= upper_bound)]
```

Python 직접
구현

Data Preprocessing with Python

- Introduction
- Data Cleaning
 - Solutions for Missing Values / Data Inconsistency / Outliers
- Table Integration
- Data Transformation
 - Data Scaling
 - Vector Normalization
- Data Reduction & Resampling
 - Row & Column Reduction
 - Imbalanced Data
- Data Discretization
 - Encoding

Table Integration (테이블 통합)

■ 목적

- Machine Learning 을 위해서는 여러군데 나누어져 있는 Data Table들을 하나로 통합
- Data Table의 종류: Numpy 2D array, Pandas DataFrame, DBMS Relations

- 옆으로 테이블 결합: 동일한 key로 묶을 수 있는 데이터를 하나의 table로 만듦 (Database join의 느낌)

index	A	B
1001	5	6
1002	7	8

index	C	D
1001	10	11
1002	12	13



index	A	B	C	D
1001	5	6	10	11
1002	7	8	12	13

- 아래위로 테이블 붙임: Homogeneous한 테이블을 아래위로 합쳐 하나의 table로 만듦 (Python append() 느낌)

index	A	B
1001	5	6
1002	7	8

index	A	B
1003	3	4
1004	2	3



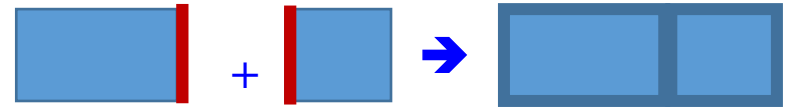
index	A	B
1001	5	6
1002	7	8
1003	3	4
1004	2	3

Numpy에서 Table 통합

- Concatenating ndarray Objects

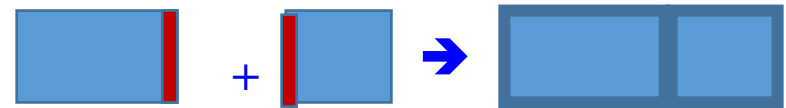
Concatenate 2D Array Objects

```
a = np.array([[1,2], [3,4]])  
b = np.array([[5,6]])  
np.concatenate((a,b))  
→ array([[1, 2],  
         [3, 4],  
         [5, 6]])
```

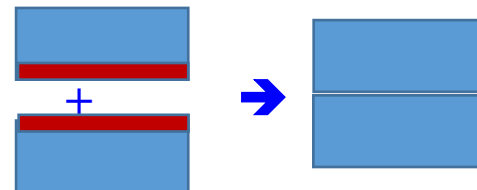


- Various Stacking with Matrices (in np module)

- `np.hstack()` : horizontal stacking
 - row숫자가 같은 matrix들에만 적용



- `np.vstack()` : vertical stacking
 - column숫자가 같은 matrix들에만 적용

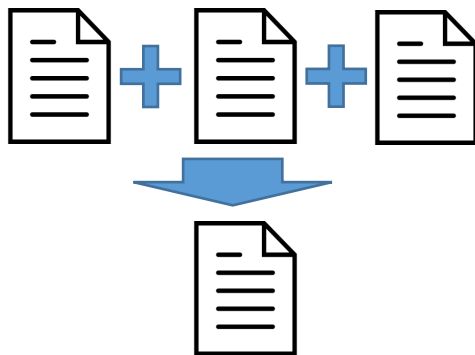


Pandas에서 Table 통합

- 여러개의 Pandas DataFrame들의 통합



- Heterogeneous Table들의 결합에 주로 쓰임
- 예: User 테이블, Purchase 테이블, Basket 테이블
- `pd.merge()` & `df.join()`
- 직관: DB-Join 유사



- Homogeneous Table들의 붙임에 주로 쓰임
- 예: xxx_1.csv, xxx_2.csv, xxx_3.csv
- `pd.concat()`
- 직관: DB-Union 유사, Python `append()` 유사

Data Preprocessing with Python

- Introduction
- Data Cleaning
 - Solutions for Missing Values / Data Inconsistency / Outliers
- Table Integration
- Data Transformation
 - Data Scaling
 - Vector Normalization
- Data Reduction & Resampling
 - Row & Column Reduction
 - Imbalanced Data
- Data Discretization
 - Encoding

Data Scaling (데이터의 단위 통일화 & 데이터 크기조정)

- 컬럼마다 데이터 측정 단위가 다를 경우 모델에 부정적 영향
 - 모든 컬럼의 단위를 일정하게 하는 통일화 작업 필요
 - 데이터값이 너무 크거나 작으면 ML 학습과정에서 0으로 수렴 혹은 ∞ 로 발산
- Scaling 작업: 모든 데이터에 선형 변환을 적용하여 데이터의 범위를 0~1로 축소
- Scaling의 효과
 - 기계학습의 성능(prediction) 향상, 수렴 속도(performance) 향상, Outlier의 영향 감소

몸무게	키	월급
72	182	2500000
55	155	2000000
50	167	15000000
88	192	9000000



몸무게	키	월급
0.65	0.82	0.41
0.53	0.56	0.34
0.51	0.61	0.99
0.71	0.91	0.89

직관: 어떤 column (feature)가 다른 column (feature)를 지배하지 못하게 하는 작업

** 각 column마다 사연이 있고, 의미가 다르지 않나? 그런데 데이터를 무조건 0~1로 단위를 통일해도 되나?

➔ 기계학습 Model은 각 column들의 사연(의미)을 알수가 없으므로 Scaling으로 단위를 통일하는게 바람직!

Data Scaling in Sklearn.preprocessing submodule

1. **Standard scaling**: 평균값 & 표준편차값을 이용해서 데이터를 ($\mu=0, \sigma=1$)인 데이터 변환

(범위 표준화 : Standardized Score (표준점수), 표준 점수화, Z score 변환) $\tilde{x}_i = \frac{x_i - \text{mean}(x)}{\text{std}(x)}$

2. **Min-Max scaling**: 데이터값의 범위를 (0 ~ 1)로 데이터 변환

$$\tilde{x}_i = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

(범위 정규화)

3. **Max-Abs scaling**: 데이터값의 범위를 (-1 ~ 1)로 데이터 변환

$$\tilde{x}_i = \frac{x_i}{\max(|x|)}$$

(최대 절대값 scaling: 데이터의 절대치가 1을 넘지 않게)

4. **Robust scaling**: median값 & IQR값을 가지고 Scaling

$$\tilde{x}_i = \frac{x_i - \text{median}(x)}{Q3(x) - Q1(x)}$$

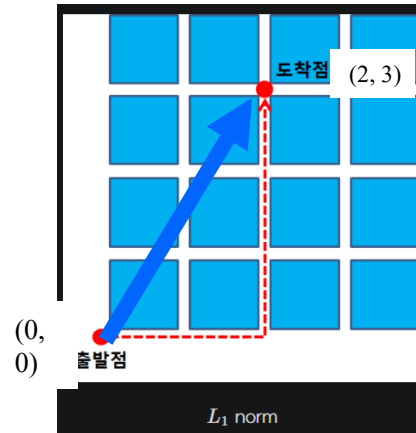
(중앙값-4분위수 Scaling: Outlier 영향 최소화)

- Scikit-learn에서 ML작업 수행 전에 Data Scaling을 수행
- SKLearn Module의 function
- SKLearn Module의 class

Scaling 방식	Function	Class
Standard Scaling	<code>scale(x)</code>	<code>StandardScaler</code>
Min-Max Scaling	<code>minmax_scale(x)</code>	<code>MinMaxScaler</code>
Robust Scaling	<code>robust_scale(x)</code>	<code>RobustScaler</code>
Max-abs Scaling	<code>maxabs_scale(x)</code>	<code>MaxAbsScaler</code>

Vector Norm

- **L1 Norm:**
Manhattan norm
or Taxicab norm

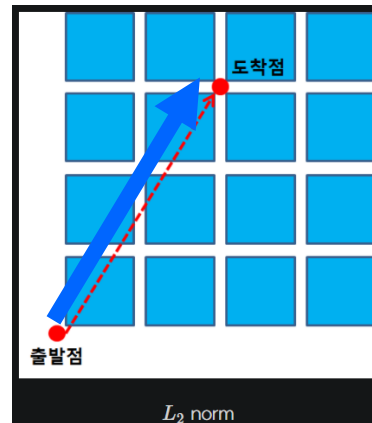


$$x = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

이 벡터의 L_1 norm은 5

$$\|x\|_1 = |2| + |3| = 5$$

- **L2 Norm:**
Euclidean Norm
출발점에서 도착점까지의
거리를 직선거리로 측정



- $L_2 \text{ Norm} = \sqrt{\sum x_i^2} \quad (= \|x\|)$
- $x = (3, 4)$
- $L_2 \text{ norm of } x = \sqrt{3^2 + 4^2} = 5$

L2 norm 기반 Normalization

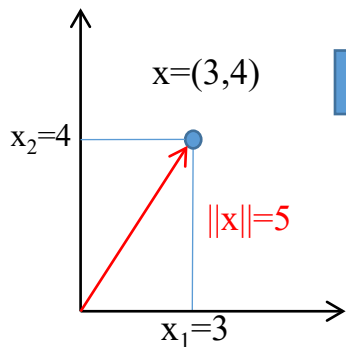
- L2 Norm: 한 Row에 있는 데이터 값들을 각각 자승하여 합한후에 square root를 적용

Given $x = x_1, x_2, \dots, x_n$ **norm**(x) = $\sqrt{\sum x_i^2}$ (= $\|x\|$)

- L2 Normalization: 한 Row에 있는 개별 데이터를 그 Row의 L2 Norm값으로 나눠주는 작업

$$\hat{x}_i = \frac{x_i}{\|x\| \text{ (= norm)}}$$

- Normalization한후에 x 값들의 norm = $\sqrt{\sum \hat{x}_i^2} = \|\hat{x}\| = 1$



$$\hat{x}_1 = 3 / 5 = 0.6$$

$$\hat{x}_2 = 4 / 5 = 0.8$$

$$(\hat{x}_1, \hat{x}_2) = (0.6, 0.8)$$

$$\|\hat{x}\| \rightarrow \sqrt{0.6^2 + 0.8^2} \rightarrow 1$$

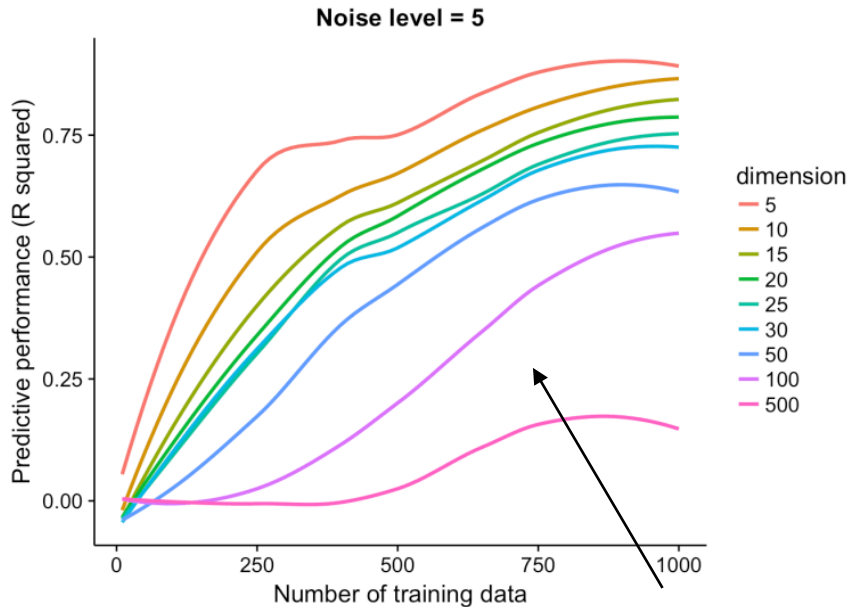
- L2 normalization이후에는 원점(0,0)에서 모든 점까지 거리가 1로 통일된다
- L2 Normalized 된 점 $(\hat{x}_1, \hat{x}_2)$ 들의 분포는 original 점 (x_1, x_2) 들의 분포와 다르게 된다

Data Preprocessing with Python

- Introduction
- Data Cleaning
 - Solutions for Missing Values / Data Inconsistency / Outliers
- Table Integration
- Data Transformation
 - Data Scaling
 - Vector Normalization
- Data Reduction & Resampling
 - Row & Column Reduction
 - Imbalanced Data
- Data Discretization
 - Encoding

Curse of Dimensionality (데이터 차원의 저주)

- Need to reduce dimension of data (size of matrix)
- Column 숫자 & Row 숫자가 이 많아지면 요구되는 계산은 기하급수적으로 증가
- 학습에 많은 시간, 성능에도 나쁜 영향



- Need to remove **duplicate rows**
 - In Numpy Object or Pandas DataFrame Object
- Need to remove **unnecessary columns**
 - In Numpy Object or Pandas DataFrame Object



차원이 지나치게 크면 데이터가 많아지고 기계 학습의 정확도 성능도 저조해진다

Duplicate Rows, Duplicate Columns, Unnecessary Columns

Numpy Object

[Duplicate Row]

```
sample = np.array( [ [1,2,3,4],  
                    [1,2,3,4],  
                    [5,6,7,8],  
                    [1,2,3,4],  
                    [3,3,3,3],  
                    [5,6,7,8] ] )
```

[Duplicate Column]

```
sample = np.array( [ [1, 2, 1, 4],  
                    [1, 2, 1, 4],  
                    [5, 6, 5, 8]  
                    ] )
```

Pandas Object

[Duplicate Row]

	A	B
0	1	apple
1	2	banana
2	3	apple
3	4	orange
4	2	banana

[Duplicate Column]

	A	B	C
0	1	apple	1
1	2	banana	2
2	3	apple	3
3	4	orange	4

[Unnecessary Column]

	A	B	C
0	1	apple	5
1	2	banana	6
2	3	apple	7
3	4	orange	8

	오존	이산화질소	오존+0.123
일			
1	0.008792	0.038958	0.131792
2	0.023792	0.017292	0.146792
3	0.007333	0.039083	0.130333
4	0.012792	0.033875	0.135792
5	0.023917	0.014042	0.146917
6	0.011792	0.027583	0.134792
7	0.002250	0.052208	0.125250
8	0.005625	0.045125	0.128625
9	0.003292	0.050083	0.126292
10	0.010167	0.054875	0.133167
11	0.020727	0.031773	0.143727
12	0.015750	0.018167	0.138750
13	0.004042	0.040500	0.127042
14	0.014167	0.022583	0.137167
15	0.006458	0.037458	0.129458
16	0.003000	0.046458	0.126000

Perfect Correlation Columns

Linear Algebra Operations for Data Reduction

Eigen Decomposition

$$\mathbf{A} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^{-1}$$

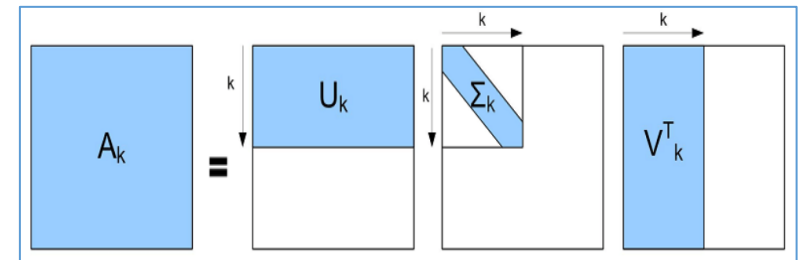
$\mathbf{A}$ is represented by a 3x3 grid.

$\mathbf{Q}$ is a matrix with columns $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$, labeled "Eigen vectors of $\mathbf{A}$ ".

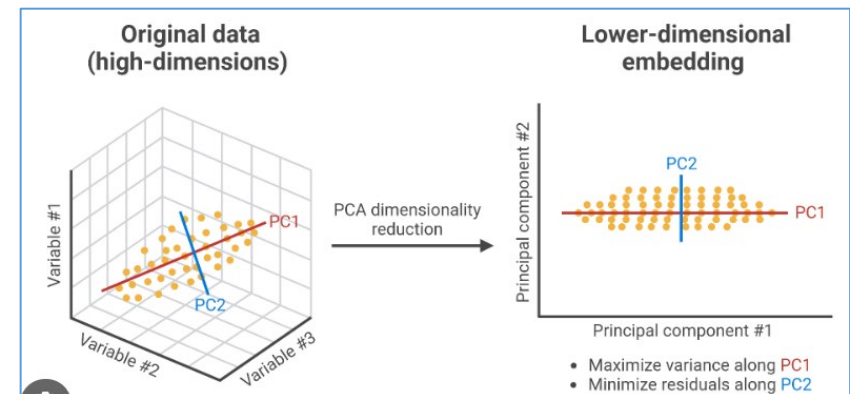
$\mathbf{\Lambda}$ is a diagonal matrix with entries $\lambda_1, \lambda_2, \lambda_3$, labeled "Eigen values of $\mathbf{A}$ ".

$\mathbf{Q}^{-1}$ is a matrix with columns $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$, labeled "Eigen vectors of $\mathbf{A}$ ".

Singular Value Decomposition (SVD)



Principal Component Analysis (PCA)



Linear Algebra 수업 들어야 함!

Imbalanced Data

■ 학습용 데이터의 클래스 비율이 너무 차이가 나는 경우

- Majority Class를 예측하는 모델의 정확도가 너무 높아져 성능 판별 어려움

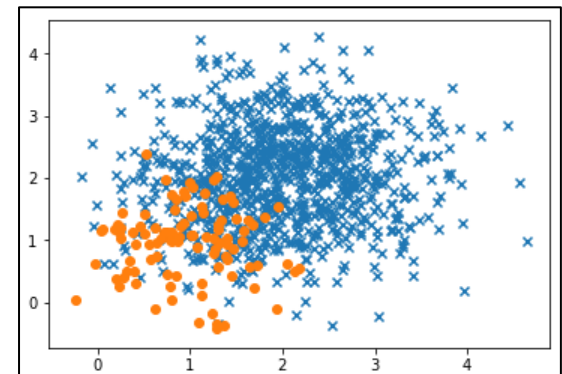
■ Issue

- 두종류의 다각형 Class A와 Class B의 비율이 99:1 인 Data Set이 있다
- Test Data는 학습데이터에서 일정량을 Class A와 Class B의 비율대로 확보
- Machine Learning Model M은 무조건 Class A라고 예측 대답을 하는 저질 Classifier
 - 그러나 M의 성능은 99%! (Class B는 전혀 맞추지 못하는 저질 Classifier인데....)

■ Sampling을 사용하여 데이터의 비율 조정

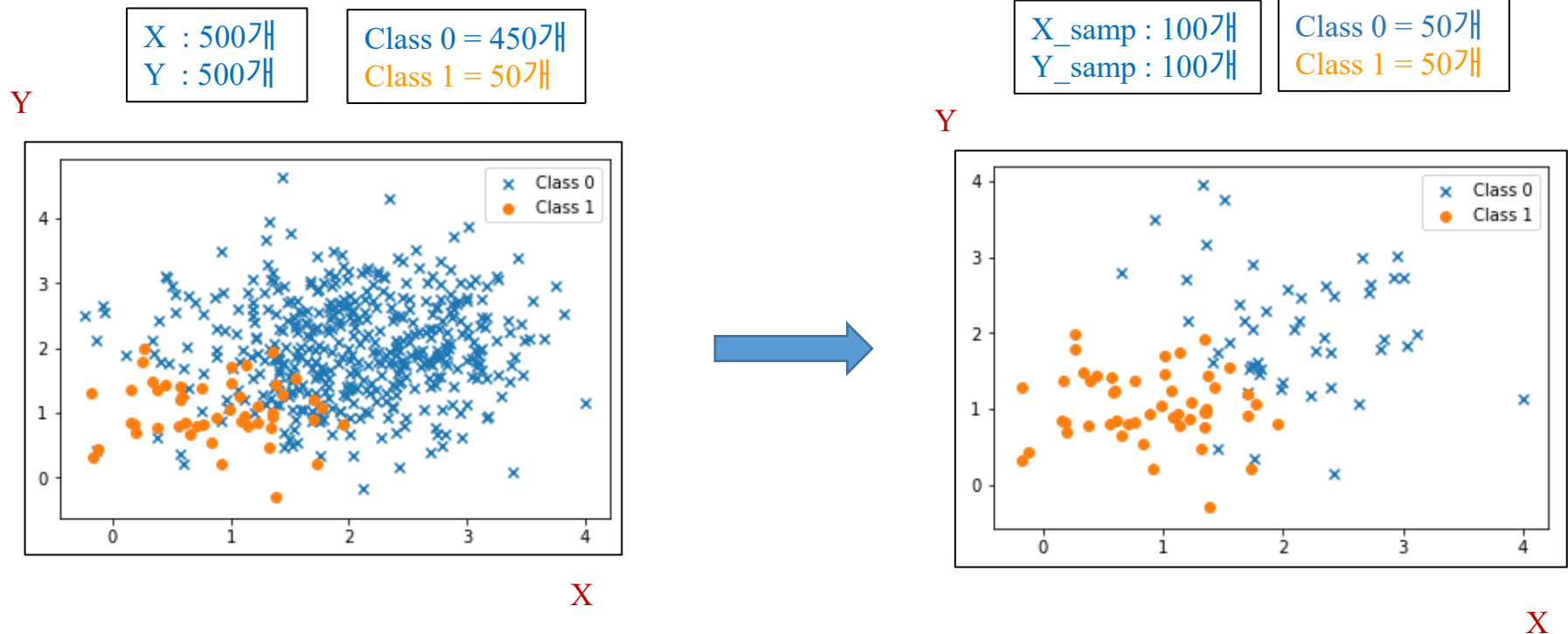
- **Oversampling**: Minority Class에서 Data를 복제 또는 증폭
- **Undersampling**: Majority Class 에서 Data의 일부를 제거

- imbalanced-learn module 설치 필요
- 설치후에는 일반적으로 imblearn 별칭으로 import
- `>>> import imbalanced-learn as imblearn`



UnderSampling 방법

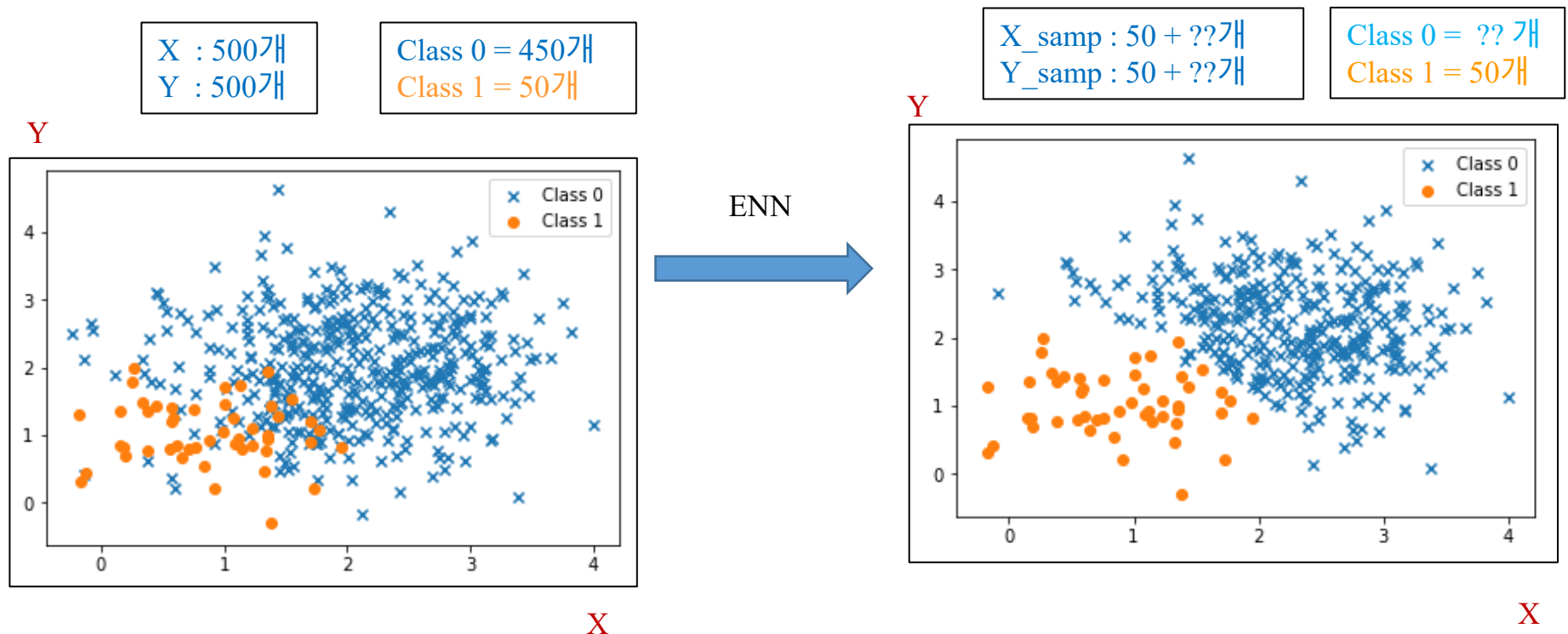
RandomUnderSampler Class from `imblearn.undersampling`



Blue point들중에 Orange point 개수 만큼
만 남기고 random하게 Blue point 를 제거

UnderSampling 방법

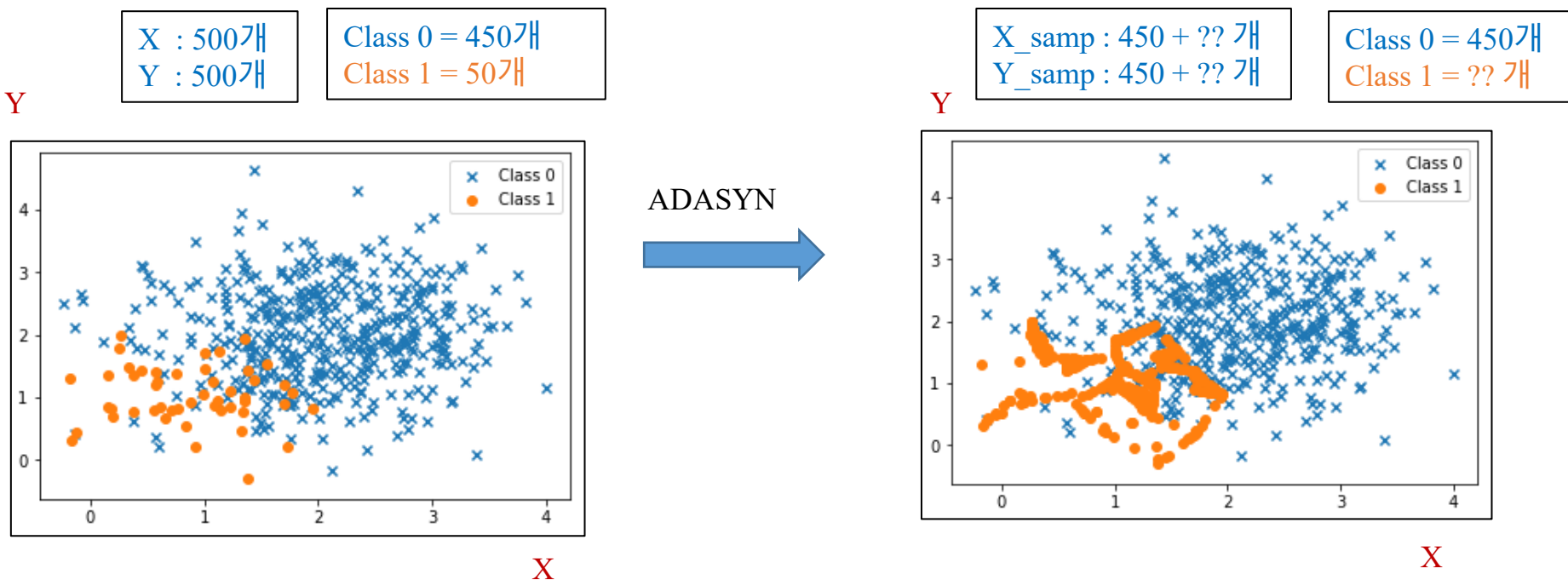
EditedNearestNeighbors Class from `imblearn.undersampling`



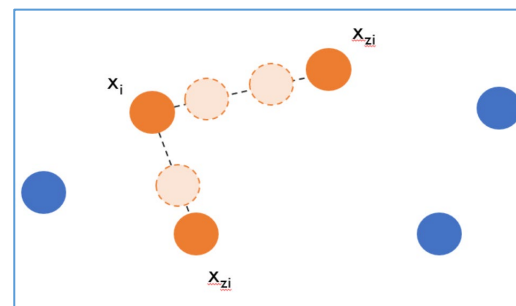
Orange point 주변에 k개의 nearest point
들중에 Blue point X 가 있으면 X를 제거

OverSampling 방법

ADASYN Class from imblearn.oversampling



- 특정 orange point와 인접한 k개의 orange point중 임의선택된 point간의 직선위에 가상의 orange point 생성



Data Preprocessing with Python

- Introduction
- Data Cleaning
 - Solutions for Missing Values / Data Inconsistency / Outliers
- Table Integration
- Data Transformation
 - Data Scaling
 - Vector Normalization
- Data Reduction & Resampling
 - Row & Column Reduction
 - Imbalanced Data
- Data Discretization
 - Encoding

Data Discretization (데이터 이산화)

- Data Discretization: 데이터를 이산적인 구성요소로 변환하는 과정
 - 범주형 값이나 텍스트 정보를 Numeric Data 로 변환
 - 단순 Numeric Data를 ML에 적합한 형식의 데이터로 변환 (Encoding)
 - % 로 표현된 데이터 (예, 확률) : 0 ~ 1 사이의 값으로 변환
- Preprocessing의 마무리 단계에서 수행
 - Machine Learning platform으로 데이터를 보내는 단계
- Example

- Data Encoding for Data Part
- Label Encoding for Label Part

Season	Season
Spring	1
Summer	2
Fall	3
Winter	4
Summer	1

단순 숫자 변환



Spring + 1 = Summer?
불분명하고 애매한면이 있다

도명	Season
경기도	1
충청도	2
경상도	3
전라도	4
강원도	5

단순 숫자 변환



경기도 + 1 = 충청도?
불분명하고 애매한면이 있다

ML에 적합한 Encoding

- 기본 Encoding: 범주형 값이나 텍스트 정보를 단순 숫자 (Simply Digitizing) 변환
- 문제점: 범주형 정보나 텍스트 정보에 없는 linearity 가 형성
 - 형성되는 Machine Learning Model에 영향을 줄수 있다!
- 기계학습(ML)에 적합한 추가 Encoding이 필요!



Data Encoding vs. Label Encoding

Data

Label

Age	Sex	BMI	ABP	1년뒤진행
28	남	21.5%	80..120	당뇨
72	여	19%	70..115	당뇨전단계
55	남	3%	60..90	정상



Data

Label

Age	Sex	BMI	ABP	1년뒤진행
0.038	0.051	0.062	0.022	2
-0.002	-0.045	-0.051	-0.026	1
0.085	0.051	0.044	-0.006	0

- **Sklearn.preprocessing submodule**

- OneHotEncoder class
- Binarizer Class
- KBinsDiscretizer Class
- (many other encoders)

Data Encoding

- LabelEncoder Class
- LabelBinarizer Class

Label Encoding

Label = Target Data = 정답 = Y값

- 수강생 여러분!!! 수고했습니다!
- 주말동안 Numpy, Pandas, Matplotlib, Sklearn의 인터넷강의들을 수강하고 실제 코딩실습을 많이 하길 바랍니다
- Good Luck!